FIR 滤波器 Zynq 软硬件协同实现报告

摘要

本作品基于 Zynq‑7000 异构平台实现 51 抽头 FIR 数字滤波器软硬件协同设计。PL 端使用 FPGA 硬件实现 FIR 滤波加速逻辑，借助 AXI‑DMA 完成 PS 与 PL 之间高速数据交互；PS 端 Cortex‑A9 处理器实现软件版本 FIR 作为性能参照。通过 Python 完成激励信号生成、理论 Golden 结果计算，完成板级功能验证与性能对比测试。

测试采用 1000 点正弦、混频输入信号，FPGA 硬件输出与 Python 参考模型结果逐点比对，最大绝对误差不超过 3.0×10^{-6}，功能正确性得到验证。ARM‑O0 编译纯计算耗时约 2469 μs，FPGA+DMA 端到端处理耗时约 96 μs，对照比可达约 25.85 倍。本设计充分发挥 FPGA 并行计算优势，可为嵌入式实时数字信号滤波系统提供软硬件协同实现参考，整套设计完成仿真、板上验证与性能评估。

第一部分　作品概述

1.1 功能与特性

本项目面向嵌入式端固定长度数字信号滤波，以 1000 点正弦、混频序列验证 51 抽头 FIR 低通实现。FPGA 设计方法参见文献[1]，Zynq 架构参见[2]，FIR 原理与 DMA 机制分别参见[3][4]。PS 负责控制与软件基准，PL 执行滤波及 DMA 搬运，PC 进行离线验证；当前未接入外部 ADC 或在线模型推理。

FPGA 开发流程参考文献[5]。项目保留 ARM 软件 FIR 与 PL 硬件通路，完成核心板实测，并利用 Python 生成参考、解析输出、计算误差与绘制波形。工程贡献是固定参数滤波器的软硬件集成、双通道数据传输及可核对的验证证据，不将成熟 FIR 算法、对称结构或 DMA 机制宣称为原创。

1.2 应用领域

FIR 滤波器是数字信号处理基础模块，本设计可应用于多种实时信号采集处理场景。在音频处理领域，可用于音频信号降噪、频段筛选；在传感器信号处理中，能够对采集到的振动、电压信号做平滑滤波，抑制高频噪声干扰。

在通信预处理环节，可完成信号的基带滤波，滤除带外干扰分量。本设计基于 Zynq 嵌入式平台，体积紧凑，无需上位机持续介入，适合嵌入式端本地化实时信号处理，可拓展应用于便携式检测设备、嵌入式监测终端等场景，为各类信号处理系统提供硬件加速滤波能力。



图 1 应用示意图

1.3 主要技术特点

采用 Zynq 异构软硬件协同架构，PS 负责控制、任务调度，PL 利用并行流水线实现 FIR 硬件加速，发挥 FPGA 并行计算优势。使用 AXI- DMA 实现 PS 与 PL 之间大批量数据高速搬运，减少 CPU 干预开销。

搭建完整 Python 验证工具链，统一生成输入激励与理论 Golden 参考，对 FPGA 板上输出 bin 文件解析，自动计算最大绝对误差、RMSE，绘制波形与误差曲线，实现自动化验证。同时保留 ARM 软件 FIR 作为性能参照，完成板上实测耗时统计，区分纯计算耗时与端到端完整通路耗时，客观评估硬件加速收益，仿真、离线验证、板级实测流程完整闭环。

1.4 主要性能指标

指标	正弦	混频
抽头数	51	
样本数	1000	
ARM Cortex-A9 -O0 纯计算耗时	2469.3239 μs	2468.7209 μs
FPGA+DMA 端到端耗时	96.2880 μs	95.5110 μs
耗时对照比	约 25.65 倍	约 25.85 倍
最大绝对误差	不超过 3.0×10^{-6}	

说明：ARM 计 FIR 纯计算；FPGA 计输入准备、缓存维护、DMA、FIR 运算及传输完成等待的端到端耗时。两者为耗时对照比，非严格同口径加速比。

1.5 主要创新点

1. 集成 51 抽头 FIR、双通道 DMA 与 PS 控制，形成可核对的软硬件协同通路。
2. 保留 ARM 软件参照及板测 BIN，结合 Python 逐点验证，明确参考模型与误差口径。
3. 区分纯计算和端到端计时，并结合模块及系统资源报告客观说明实现边界。

1.6 设计流程

首先完成 FIR 算法理论建模，使用 Python 生成测试激励与理论输出；其次完成 FPGA 端 FIR 硬件逻辑与 AXI‑ DMA 接口设计；再编写 ARM 端软件 FIR 参考程序；之后进行仿真验证，随后下载比特流到 Zynq 核心板开展板级实测；采集耗时与输出数据，利用 Python 工具做误差分析与性能评估，最后整理报告。

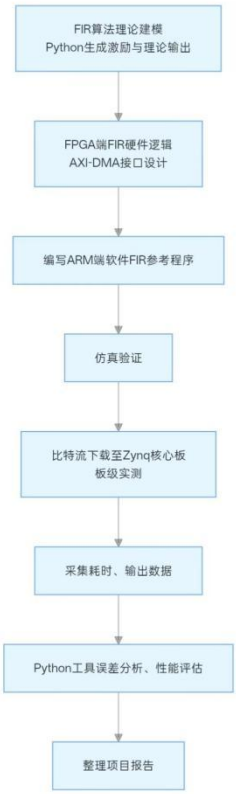


图 2 设计流程图

第二部分　系统组成及功能说明

2.1 整体介绍



图 3 系统整体框图

系统分为 PS 处理子系统、PL 硬件加速子系统、PC 端 Python 验证工具链三大部分。

PS 子系统：Cortex‑ A9 处理器，运行 ARM 软件 FIR 算法，负责下发任务、数据缓存管理，启动 DMA 传输，采集运行耗时。

PL 子系统：包含 FIR 滤波硬件模块、AXI‑ DMA 控制器，接收 PS 发送的输入数据，完成并行滤波运算，将结果回传给 PS。

PC 端 Python 工具链：生成测试信号、输出理论 Golden 基准；解析从板上导出的 bin 结果文件，做误差计算、绘图分析。

PS 与 PL 通过 AXI 总线、AXI‑ DMA 完成数据交互；PC 端通过文件方式完成测试数据的输入输出。

PS 承担调度、缓存维护及软件参照，便于控制和调试；PL 利用固定系数与流水结构完成并行滤波，DMA 负责批量搬运；PC 提供离线观测和 Golden 比对。该划分保留灵活控制并分担计算，但端到端仍有 CPU 开销，现有非同口径结果不证明硬件在所有场景下不可替代。

2.2 硬件系统介绍

2.2.1 硬件整体介绍；

硬件平台为 Zynq‑ 7000 核心板，集成 PS（Cortex‑ A9 双核）与 PL FPGA 逻辑资源。PS 负责控制与软件参考运算；PL 实现 FIR 滤波器硬件逻辑，搭配 AXI‑ DMA 实现 PS‑ PL 之间高速批量数据传输。整个设计无需额外外设，利用核心板片上资源完成全部逻辑与数据交互。

环境为 Windows 及 Vivado、Vivado HLS、Xilinx SDK 2019.1，器件 xc7z020clg400‑2。板卡供电、极性、启动拨码和 JTAG 跳线须按实物手册核对；精确商品板型及供电/拨码清单尚未完整归档，不能套用其他板卡参数。

2.2.2 机械设计介绍

本设计无定制机械结构，直接使用商用 Zynq 核心板，无额外 CAD 机械设计。

2.2.3 电路各模块介绍

本设计全部使用 Zynq 片内逻辑与片上总线，无外部扩展电路。关键信号为 AXI-DMA 读写通道信号，实现 PS 内存与 PL FIR 模块之间输入输出数据流搬运。

FIR 按 $y[n]=\Sigma(k=0\cdots50)h[k]x[n-k]$ 计算，$n-k<0$ 时输入置零；每次调用清零 history。固定系数对称，接口传输 float32 位模式，核心内部为定点计算。源码采用 history 完整分区、UNROLL 及样本循环 PIPELINE；既有 HLS 报告显示样本循环 achieved II=1，不据此推算 DMA 端到端时延。

数据通路如图 4(a)所示。MM2S 从 DDR 中的 TxBuffer 读取输入，经 M_AXIS_MM2S 进入 FIR 的 in_r；FIR 完成滤波后，由 out_r 向 S_AXIS_S2MM 输出，S2MM 将结果写回 DDR 中的 RxBuffer。两端均采用 AXI4-Stream 接口。

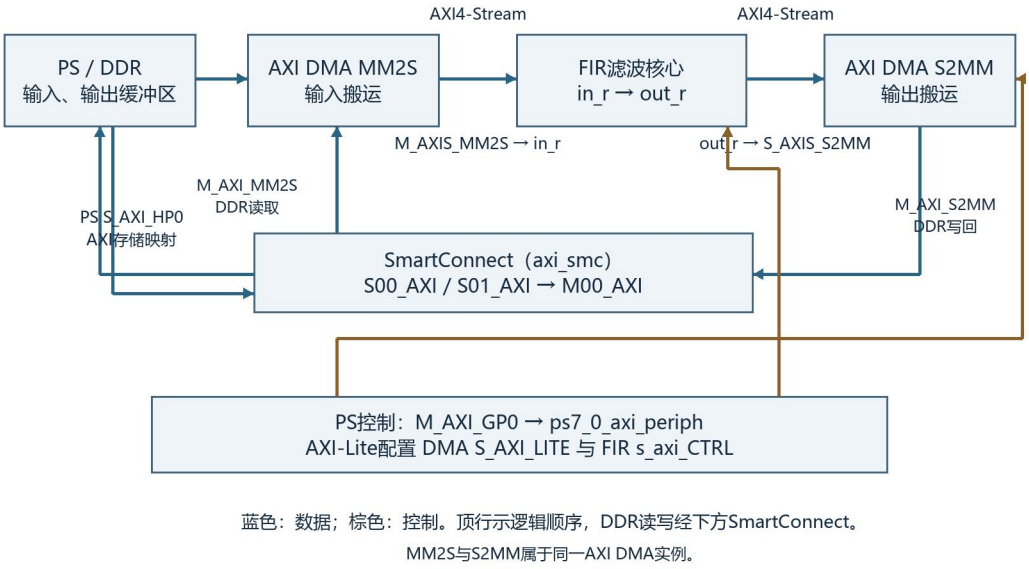


图 4(a) FIR 局部数据通路与控制接口

DMA 的 M_AXI_MM2S、M_AXI_S2MM 分别接入 SmartConnect 的 S00_AXI、S01_AXI，再经 M00_AXI 连接 PS 的 S_AXI_HP0 访问 DDR。PS 通过 M_AXI_GP0 和 ps7_0_axi_periph，以 AXI-Lite 配置 DMA 及 FIR 控制寄存器。MM2S 负责输入搬运，S2MM 负责输出写回；这些连接与现有 Block Design 一致[2,4]。

AXI4-Stream 采用 32 位 TDATA 及 TVALID/TREADY 握手，TLAST 标记末样本；N=1000 对应 4000 字节。AXI-Lite 只用于寄存器控制，DDR 访问为 AXI 存储映射。数据长度、末样本标记和缓冲区必须一致。

整套系统经过 Vivado 布局布线实现，图 5 所示的全系统实现阶段资源为 Slice LUTs 7084 个、Slice Registers 8010 个、Block RAM 2 个、DSP48E1 25 个。FIR IP 独立综合（OOC）资源为 3053 个 LUT、25 个 DSP、0 个 BRAM；该数据来自 FIR IP 自身综合报告，不是顶

层 wrapper 综合报告。25 个 DSP 全部由 FIR 核心占用，2 个 Block RAM 分别属于 AXI DMA 的 MM2S 和 S2MM 数据 FIFO；FIR 核心本身不使用 Block RAM，其历史数据存储映射为 LUTRAM。FIR 共有 26 个乘法表达式，包括 25 组对称配对和 1 个中心抽头。由于 h[0]=0，第 0 组对称乘法结果恒为 0，被 HLS 常量传播消除；剩余 24 组对称乘法及 1 个中心抽头，共 25 个乘法器，对应 25 个 DSP48E1，其中中心抽头 h[25]占用 1 个 DSP。

图 6 为布线后时序汇总：WNS=8.855 ns、WHS=0.036 ns、TNS=0、THS=0。建立、保持及脉宽检查的所有端点均满足时序要求，没有时序违例；其中保持时序余量较小。PL 实际工作时钟为 PS FCLK_CLK0，配置为 50 MHz（周期 20 ns），布线后时序报告也按 20 ns 进行分析。HLS 综合目标时钟为 100 MHz（周期 10 ns），属于模块综合估计阶段的目标约束，不代表集成后 PL 的实际工作频率。

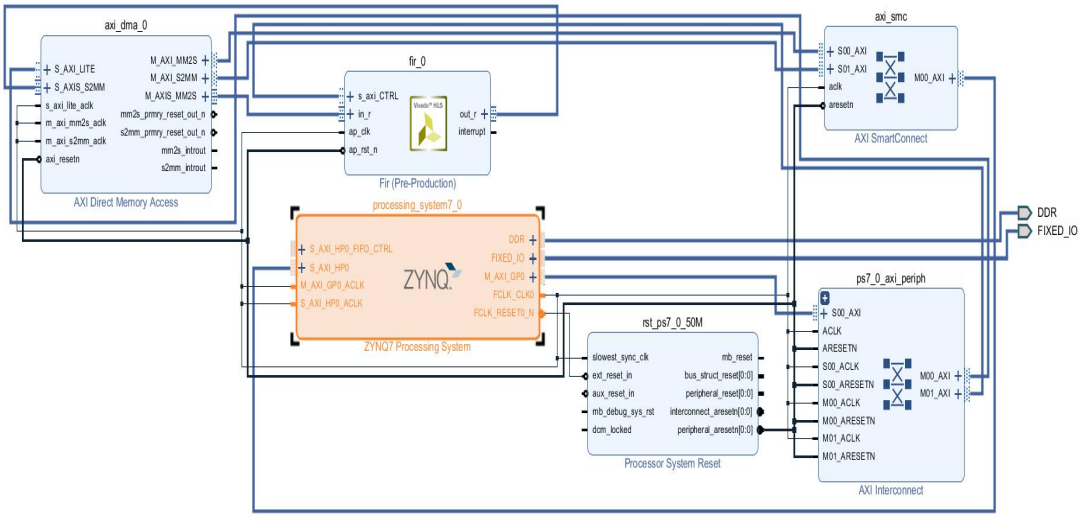


图 4(b) Vivado Block Design（原工程视图）

Name	Slice LUTs (53200)	Slice Registers (106400)	F7 Muxes (26600)	Slice (13300)	LUT as Logic (53200)	LUT as Memory (17400)	Block RAM Tile (140)	DSPs (220)	Bonded IOB (125)	PHY_CONTROL (4)	BUFIO (16)
fir_system_wrapper	7084	8010	1	2909	6467	617	8010	2	25	130	1
fir_system_i (fir_system)	7084	8010	1	2909	6467	617	2	25	0	0	0
axi_dma_0 (fir_system_	1256	1826	0	633	1168	88	2	0	0	0	0
axi_smc (fir_system_ax	2250	3040	0	980	1784	466	0	0	0	0	0
fir_0 (fir_system_fir_0_(	3050	2454	1	1092	3049	1	0	25	0	0	0
processing_system7_0	0	0	0	0	0	0	0	0	0	0	0
ps7_0_axi_periph (fir_s	513	657	0	284	452	61	0	0	0	0	0
rst_ps7_0_50M (fir_sys	16	33	0	10	15	1	0	0	0	0	0

图 5 工程资源占用统计图

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	8.855 ns	Worst Hold Slack (WHS):	0.036 ns	Worst Pulse Width Slack (WPWS):	8.870 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	26520	Total Number of Endpoints:	26520	Total Number of Endpoints:	9162

All user specified timing constraints are met.

图 6 时序汇总结果

2.3 软件系统介绍

2.3.1 软件整体介绍

软件分为两部分：Zynq 板端嵌入式软件，PC 端 Python 验证软件。

板端：ARM C 程序，包含软件 FIR 参考实现、计时逻辑、DMA 控制、数据缓存管理，完成输入数据准备，分别执行 ARM 软件 FIR 与调用 FPGA 硬件通路，记录各段耗时，输出二进制结果文件。

PC 端 Python：开发时使用 signal_generate.ipynb 与 fir_hw_verify.ipynb 生成激励/参考、解析 BIN 并绘图（图 7）。原 Notebook 未随包保留，现已提供 04_Test_Evidence/python_verify 中的同名.py 导出脚本，以及 05_Docs/compare.py 独立复算程序。前者依赖 numpy/scipy/matplotlib，后者仅使用 Python 标准库。

```
# ==================== 1. 小端float32 bin文件转csv（精度改成%.9g） ====================
def bin_to_float_csv(bin_path, out_csv_path, n_sample):
    with open(bin_path, "rb") as f:
        bs = f.read()
    data = np.array(struct.unpack("<" + "f"*n_sample, bs), dtype=np.float32)
    np.savetxt(out_csv_path, data, fmt="%.9g")
    return data
```

图 7 Python 端 bin 解析

2.3.2 软件各模块介绍

1. 信号生成模块：输入参数（采样点数、滤波器抽头数），输出正弦、混频输入序列、滤波器系数、Python 理论滤波输出 Golden 数据。

2. bin 解析转换函数：读取板上小端 float32 二进制 bin 文件，转换为浮点数组并保存 csv，输出为可比对数据集。

3. 绘图比对函数：接收理论值、FPGA 实测值、误差数组，绘制完整波形与局部放大波形、误差曲线。

4. 性能测试模块：加载实测耗时参数，计算最大误差、RMSE、ARM- FPGA 耗时对照比，输出量化测试结果。

软件自顶向下由 main()组织两类输入的板端测试，再由 PC 端离线解析和验证。主要接口如下；表中区分现有可核对代码与开发时 Notebook 验证功能。

模块 / 代码接口	主要输入	主要输出
信号准备	采样点数、频率、滤波器参数	SineInput 、 MixedInput ； h[] / ReferenceCoeffs
ARM FIR sw_fir_filter(x, n, y)	输 入 x 、 点 数 n ； ReferenceCoeffs	y； SwSineOut / SwMixedOut； 外层计时 sw_sine_us / sw_mixed_us
FPGA FIR hw_fir_run(input)	input；DMA 配置 AxiDma；FIR 控制 Fir	RxBuffer； 状态码； 外层计时 hw_sine_us / hw_mixed_us
BIN 解析 read_bin(path)	BIN 文件路径 path	float32 数组（调用处 y_hw）
误差分析 compare_one(...)	输入 x、系数 h、BIN； y_ref / y_hw	最大误差 mx、平均误差 mean、超限点数；RMSE 由原 Notebook 验证链路提供
性能分析	sw_sine_us / sw_mixed_us； hw_sine_us / hw_mixed_us	不同计时口径的耗时对照比

main()先初始化 DMA 和 FIR。ARM 路径调用 sw_fir_filter(x, n, y)，按因果 FIR 逐点计算，计时只覆盖计算调用；FPGA 路径在 hw_fir_run(input)内依次准备缓冲区、维护缓存、先启动 S2MM 接收、启动 FIR 及 MM2S 发送、轮询传输完成并使接收缓存失效，外层 XTime 计时覆盖该调用。返回 0 表示成功，负值表示传输或超时错误。

DMA 采用 Simple 模式并先接收后发送，完成后 invalidate 接收缓存；hw_fir_run 接收失败返回-5、发送失败-6、超时-7。正常全部测试完成时 test_status=4。错误检测有代码，故障注入、自动恢复和中断驱动尚未验证；新 ELF 地址须按符号重查。

板上 RxBuffer 经 SDK 导出为 BIN，导出不在 FPGA 端到端计时内。compare.py 的 read_bin(path)按小端 float32 解析，reference_fir(x,h)生成量化 Golden，compare_one(...)统计误差；该脚本不计算 RMSE 或耗时比。RMSE 与波形由队友验证链路提供，其.py 导出程序已归档，但 npy/ref_sine.csv 输入及版本锁定仍未齐全；代码存在不等于全链路已复现。

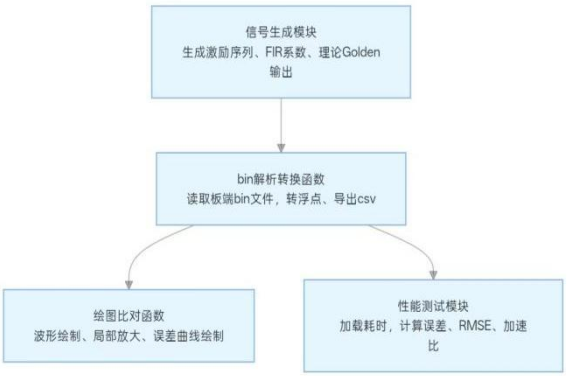


图 8(a) 原软件模块流程图

为明确 ARM 与 FPGA 两条测试路径及板外验证关系，图 8(b)补充端到端软件流程；原图 8(a)继续保留。

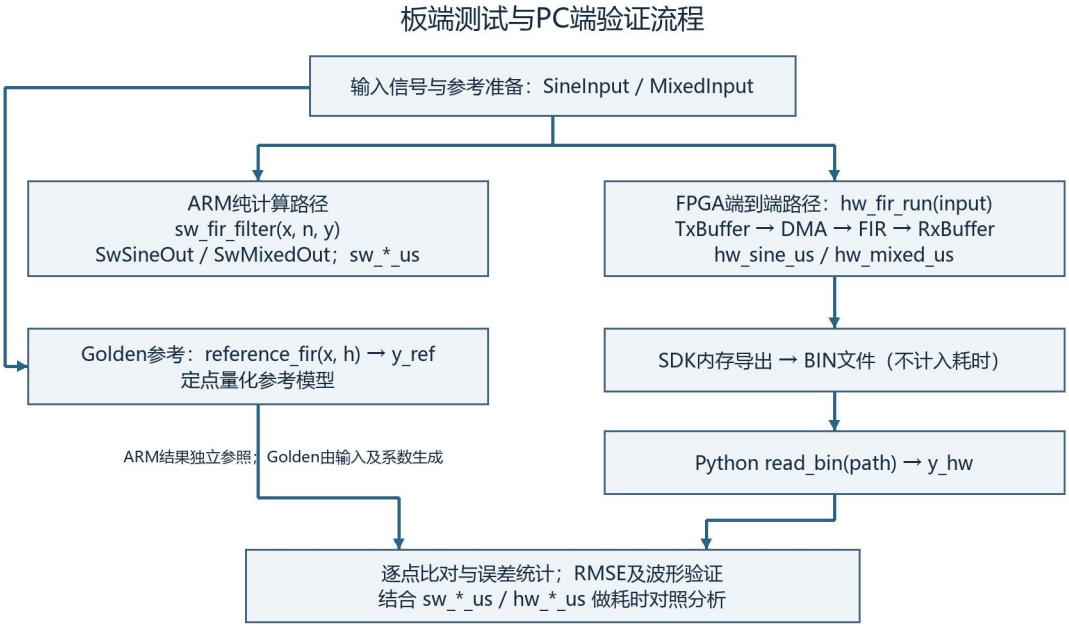


图 8(b) ARM 与 FPGA 双路径测试及离线验证流程

Zynq-7000 FIR 验证与复现 Skill

技能包位于 06_Skill/，入口为 SKILL.md；scripts/verify_fir.py 复用原 compare.py，先严格核输入和长度，再解析 BIN、计算量化 Golden 并逐点验收，输出 JSON。项目根运行：python -B 06_Skill/scripts/verify_fir.py --project .。仅离线验收自动化，硬件操作仍需人工。

功能与输入	使用与输出	验证结果与失效条件
BIN 验收：51 固定系数、输入头、1000 点小端 float32 BIN 及 dump	核 4000 字节、有限值及量化模型；输出最大误差、RMSE、超限点数和 dump 一致性。	5 组 BIN 全过、3 个 dump 逐字一致；8 项回归测试通过。缺文件/长度错/系数或位宽变更拒绝。RMSE 仅为新增离线诊断量，不替换原实验值。
DMA 排查：同批 bit/HDF/ELF、N、缓冲区和寄存器	人工依次核 flush→S2MM→FIR/MM2S→wait→invalidate、TLAST 和 4000 字节；错误码−5/−6/−7 时停止。	源码静态核对及已有正常路径证据；未做缓存/TLAST/超时故障注入或自动恢复。旧 ELF 地址、模式/缓存不匹配使前提失效。
HLS II 核查：原源码、2019.1、既有综合报告	人工查 history 完整分区、UNROLL、PIPELINE 及 achieved II；输出证据位置。	既有报告 II=1；本轮未重新综合。输入/位宽/约束变更或背压时不能直接沿用；II 不等于端到端耗时。
工程复现准备：IP、BD Tcl、SDK/BSP 依赖	按 references/reproduction.md 在独立工作区人工执行并保存新日志。	仅静态准备；完整重建、新上板与跨工程复用未验证。工具/IP 不匹配、旧.srcs/HLS 路径和缺输入须先闭环。

本次仅复算已有数据，5 组均满足最大绝对误差不超过 3.0×10^{-6}、误差大于 0.001 的点数为 0。量化参考不是位精确 ap_fixed 仿真；负例仅为临时数据副本，不代表板级故障实验。命令、输入布局、测试日志及失效条件见 06_Skill/README.md 与 TEST_RESULTS.md。

从零复现的操作与成功标志

步骤与输入	操作	产物与验收
环境/硬件：工具、Python、板卡手册与 JTAG	核 2019.1 版本/驱动，按厂家手册核供电极性、额定值、启动模式；识别目标器件。	版本和连接记录；JTAG 识别 7020。板型/供电/拨码缺证时暂停上板。
L1 已有数据：输入头、系数、5 个 BIN、dump	项目根执行 python 05_Docs/compare.py，检查所有组及实际长度，不能只看退出码。	4 组各 1000 点，误差符合 3.3 口径；3 个 dump 一致。仅证明离线复算。
HLS：src/fir.cpp、fir_test.cpp 及同级 data/	新建包外工程，top=fir、器件 7020、目标 10 ns/不确定度 12.5%；csim/csynth 并导出 IP。testbench 按 __FILE__定位数据。	新日志/IP；无 Failed checks，核 II/接口。历史 script.tcl 旧路径不可原样执行。
Vivado：ip_export 及 fir_system_bd.tcl	包外新建工程、登记 IP，source {<项目>/02_Vivado_Hardware/bd/fir_system_bd.tcl}，验证 BD、生成 wrapper、综合实现。	新 bit/HDF 及报告；核 50 MHz、HP0 和双通道。XPR 旧.srcs 依赖未闭环，不覆盖旧产物。
SDK：匹配 HDF、三工程与 xfir 驱动	新 workspace 注册驱动、按 system.mss 生成 BSP，核设备 ID/lscript.ld；-O0 只编 main.c，排除 main_v1.0.c。	ELF、链接映射及构建日志；无缺头/库/符号，记录 bit/HDF/ELF 配对。完整 L2 待验。
L3 运行：已核硬件和匹配产物	下载 bit、PS 初始化、Cortex-A9 #0 运行至 test_status=4；同次无断点运行读取四个耗时变量。	成功状态、1000 点与同批截图；固定等待时长本身不是完成证明。新批次不覆盖旧指标。
BIN：新 ELF 符号及 RxBuffer	按符号导出 4000 字节；正弦在第二次调用覆盖前保存，混频在第二次后保存。断点导出和干净计时分开。	包外新 BIN；旧 Tcl 地址不能照用；导出不计入端到端耗时。
Golden：新 BIN、同组输入/系数和验证程序	核参考口径后逐点比对。队友脚本补齐真实 npy/ref_sine.csv 输入，按 Cell 顺序运行。	日志/图；缺输入及版本未锁时不称完整链路成功。原 BIN、截图保持原样。

上述步骤在包外独立工作区执行，便于保护原成果；现阶段尚未完成归档副本的 HLS—Vivado—SDK 完整重建。供电/启动细节、工程路径、Python 中间输入及候选 ELF 仍需核实。完整操作索引见项目 README，执行成功必须以新日志和产物为证。

第三部分　完成情况及性能参数

3.1 整体介绍



图 9 Zynq 核心板实物全局照片

本作品基于 Zynq 核心板完成全部板上验证，硬件比特流、ARM 软件工程、Python 验证代码均已完成。

图 9 展示了 Zynq 核心板的 45°实物照片，图 10 展示了核心板通电状态。

3.2 工程成果

3.2.1 机械成果；

无定制机械结构，使用商用核心板。

3.2.2 电路成果；

无外部定制电路，全部为 Zynq 片上逻辑实现。

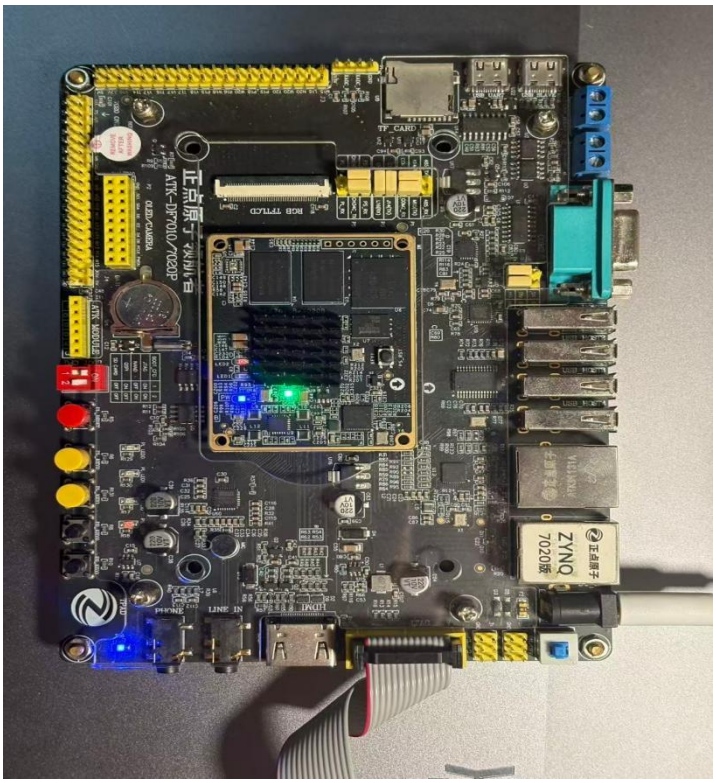
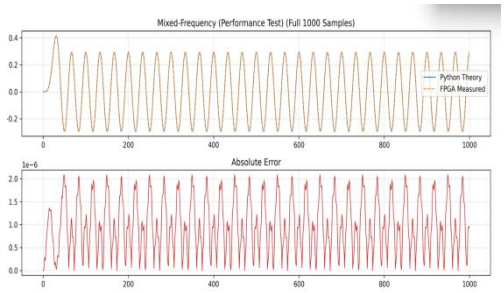


图 10 核心板通电实物照片

3.2.3 软件成果；

```
======= 混频激励（性能测试版本） =======
ARM纯计算耗时：2468.6969 μs
FPGA端到端耗时：95.4480 μs
对照比：25.86
最大绝对误差：2.08616257e-06
RMSE：1.14607496e-06
```




```
======= 正弦激励（性能测试版本） =======
ARM纯计算耗时：2469.3239 μs
FPGA端到端耗时：96.3210 μs
对照比：25.64
最大绝对误差：1.86171383e-06
RMSE：1.09561336e-06
```

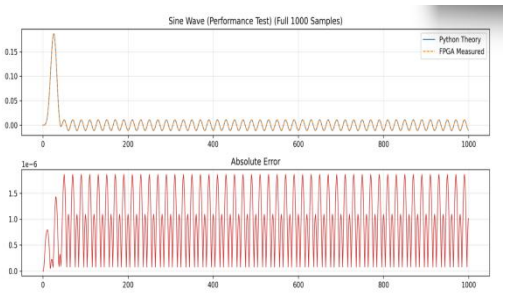


图 11 Jupyter 运行结果（另一组验证记录；最终耗时以图 12 为准）

Python 验证脚本可正常解析板上 bin 文件，输出误差指标与对比波形；ARM 工程、FPGA 比特流可正常下载至核心板运行，完成数据处理与耗时采集。 图 11 保留开发验证时的原始截图，其中耗时和误差采用该次记录的测量及参考口径，不作为本报告最终性能指标；最终耗时统一采用图 12 批次。

后期文档核查中的真实大模型协作

以下为可核对用户要求、旧稿、审计和修改记录支持的后期协作摘要，不是完整聊天记录，不证明早期算法/硬件由模型开发。AI 逐字原答、完整会话、消息 ID 及准确模型版本/时间未归档；不补造。证据 E1—E8 及提示词选段见 06_Skill/references/ai_evidence.md。

任务背景	真实问题/记录范围	AI 建议或旧稿判断	人工核验与纠错	最终采用结果	对应证据
DSP 归因：26 乘法表达式与 25DSP	用户要求按 h[0]=0 和审计§4.3 修正，并明确中心占 1DSP。	旧稿称不能确定中心映射；无法确认其为 AI 逐字原答。后期据审计整理。	用户提供审计；源码及 RTL 支持零系数消除，保留 24 对称乘法+中心。	纠正旧稿：25 乘法器对应 25DSP。未运行新综合。	E1/E2/E3；提示 P1；正文 2.2.3
性能批次：图 11/图 12 及恢复日志不同运行	用户要求保留原截图、图 11 注明其他批次，以图 12 定版。	后期建议分开原验证图与最终耗时，不能混配计数和微秒。原答未完整归档。	核两组截图、恢复日志§五勘误和 metrics，不改截图内部数据。	图 11 作另一验证记录；图 12 四变量用于定版性能。	E4/E5/E7；提示 P2；正文 3.3
计时口径：纯计算与端到端	用户要求不称严格同口径加速比。	后期建议按计时窗口说明耗时对照；不补造早期 AI 原答。	核 XTime 包围 sw_fir_filter 与 hw_fir_run；后者含准备、缓存、DMA 及等待。	采用“耗时对照比”，保留原指标及其口径说明。	E6/E7；提示 P3；正文 1.4/3.3
DMA 排查：历史故障链证据不足	本轮要求梳理错误、缓存与顺序；未找到早期故障提示词。	本轮对照源码给出检查顺序；不描述为已核实的历史诊断。	静态核缓存、先收后发、TLAST 和−5/−6/−7；已有 BIN 仅佐证正常输出。	交付人工排查流程；实际故障复现、AI 诊断及修复实测仍缺证。	E2/E6/E8；提示 P4；06_Skill/references/troubleshooting.md

参赛者负责确认事实与采纳结果，AI 负责建议和文本整理；本轮新增自动化仅验证离线文件。待补材料包括原始对话导出、回答上下文与人工判断；如申报 DMA 故障案例，还需当时错误状态、修改和修复验证。未归档多智能体工作流，不以本轮记录冒充开发历史。

3.3 特性成果

板级实测采用 1000 点正弦、混频两组测试用例。按提交包 compare.py 的定点量化参考模型复算，正弦、混频 FPGA 输出最大绝对误差分别约 2.194196×10^{-6}、2.980232×10^{-6}，统一表述为不超过 3.0×10^{-6}；大于 0.001 的误差点数均为 0。

ARM Cortex-A9 在-O0 编译条件下，正弦、混频 FIR 纯计算耗时分别为 2469.3239 μs、2468.7209 μs；FPGA+DMA 端到端耗时分别为 96.2880 μs、95.5110 μs，包含输入准备、缓存维护、DMA、FIR 运算及传输完成等待。两种不同计时口径的耗时对照比分别约 25.65 倍、25.85 倍，不作为严格同口径加速比。

Name	Type	Value
hw_sine_us	volatile double	96.28799720764808
hw_mixed_us	volatile double	95.51099723018108
sw_sine_us	volatile double	2469.323928389606
sw_mixed_us	volatile double	2468.720928407093
hw_sine_cycles	volatile u64	0x0000000000007d60
hw_mixed_cycles	volatile u64	0x0000000000007c5d
sw_sine_cycles	volatile u64	0x00000000000c8f44
sw_mixed_cycles	volatile u64	0x00000000000c8e7b
Add new express		

图 12 SDK Expressions 耗时截图（最终性能数据批次）

结果的统计范围与优化基线

性能与资源分别对应图 12 同次板测、FIR IP OOC、全系统实现及布线后时序，不能跨范围计算优化收益。资源和时钟数值见 2.2.3；HLS II=1 仅为样本循环证据。当前未留存相同器件、工具、时钟、约束、输入和计时范围的优化前实测基线，优化前 DSP、LUT、时延与收益均不填估算。ARM/FPGA 对照、IP/系统差异和 HLS/实现阶段差异都不作为优化前后实验。

复现状态分三层：L1 已有数据离线复算已有证据；L2 从源码完整重建尚待验证；L3 原定版板测有证据，但当前归档重建后的新上板重测未完成。PL 的 50 MHz 由配置及报告模型支持，未新增运行期独立频率测量；图 5/6 来源及最终 bitstream 同源性仍需团队闭环。

第四部分　总结

4.1 可扩展之处

本设计当前为固定 51 抽头 FIR 滤波器，后续可以修改硬件逻辑，实现抽头参数可配置，支持不同滤波需求；可以增加输入输出接口，对接外部 AD 采集芯片，实现真实物理信号的实时滤波。

DMA 部分可以进一步优化缓存策略，降低软件侧预处理开销，进一步压缩端到端总耗时。Python 验证工具链可以增加自动化批量测试脚本，实现多组不同长度、不同激励的一键遍历测试，提升测试效率。

后续还需补齐独立重建、真实协作记录及同条件优化基线；这些是待完成验证，不列作本次成果。

4.2　心得体会

本次 FIR 滤波器软硬件协同设计，完整走完理论建模、仿真、硬件逻辑开发、嵌入式软件编写、板级实测与验证全流程，对 Zynq PS- PL 协同工作模式有了更加深刻的理解。

最开始开发时，只关注 FIR 算法本身，忽略了计时口径的问题。ARM 只统计计算部分耗时，

而 FPGA 通路包含缓存、DMA 搬运、运算、传输等待，如果直接把两组时间直接称为加速比，结果会不够严谨。经过调试与团队沟通，明确了计时范围，改用对照比来描述性能差异，保证报告指标客观准确。

调试阶段遇到不少问题：Jupyter 验证脚本出现函数未定义报错，意识到 Jupyter 是按单元格运行顺序加载变量和函数，不是单纯从上到下解析；板上导出的 bin 文件字节序、数据位宽如果处理错误，会得到完全错乱的比对结果，必须严格按照 float32 小端格式解析。

团队分工上，A 负责 Vivado 硬件工程、ARM 板上程序，B 负责 PC 端 Python 整套验证工具，两者需要保证滤波器参数、输入信号完全对齐，否则理论参考和板上输出无法比对。多次核对滤波器抽头数、样本点数，保证两边参数统一。

通过本次实践，体会到数字系统设计中验证环节的重要性。硬件运行出现问题，很难直接看到内部信号，依靠 Python 建立 Golden 参考，做逐点误差比对，能够快速定位是算法问题、硬件逻辑问题还是数据传输问题。同时也认识到性能测试不能只看理想计算时间，需要把真实工程中的缓存、传输开销纳入考量，才能得到贴合实际工程的评估结果。

我们把字节序、执行顺序及 DMA 缓存问题归纳为输入条件、操作、验收和停止条件。已有结果支持部分正常路径，但移植和故障恢复仍待验证；后期 AI 核查需由人工回到源码和原始证据确认，不能把经验清单等同于自动化 Skill。

第五部分　参考文献

[1] EDA 先锋工作室，王诚，蔡海宁，吴继华. Altera FPGA/CPLD 设计：基础篇[M]. 2 版. 北京：人民邮电出版社，2011.

[2] AMD. Zynq 7000 SoC Technical Reference Manual: UG585, v1.15[EB/OL]. (2026-02-06)[2026-11-03]. https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM/Documentation.

[3] 胡广书. 数字信号处理：理论、算法与实现[M]. 北京：清华大学出版社，2012.

[4] XILINX. AXI DMA v7.1 LogiCORE IP Product Guide: PG021[EB/OL]. (2022-04-27)[2026-11-03]. https://www.amd.com/content/dam/xilinx/support/documents/ip_documentation/axi_dma/v7_1/pg021_axi_dma.pdf.

[5] 吴厚航. 深入浅出玩转 FPGA[M]. 北京航空航天大学出版社，2023.